ML Mini Project
Tennis Match Prediction

PES2UG24CS102-Ayushman Dey
PES2UG24CS110-Basavaraj Ramesh Dhakade

1. Problem Statement

The aim of this project is to predict the likely winner of a professional tennis match using machine learning. The system uses historical ATP match results and information about the two players to estimate which player has a better chance of winning. The prediction is made before the match, using statistics that describe player performance and previous results.

Tennis matches can be affected by several factors, including player rankings, recent form, court surface, and how players have performed in earlier matches. The project brings these details together so that a model can learn patterns from past matches and use them when predicting a new one. The output is an estimate based on historical information, since a model cannot know every condition that may affect a match.

2. Dataset Details

The dataset used by the project contains ATP match records from 1997 to 24 November 2019. The project README reports 70,251 usable matches involving 2,051 players. Each match record provides information that can be used to describe the players and identify the recorded winner. The data is prepared into a format suitable for feature generation and model training.

Matches from 1997 to 2018 were used for training, while matches from 2019 were held out for testing. Keeping a later season for testing helps measure how the model performs on matches that were not used to train it. The original dataset was not available in the main project repository, so it was obtained from an archived repository.

The reference result discussed for the project is about 71% accuracy, while the current implementation reports 64.46% on its 2019 test set. One possible reason for the difference is that the available data ends in 2019 and may be smaller or different from the data used to obtain the higher result. The results can also vary depending on how features are prepared and how the model is trained.

3. Approach

The project uses a Random Forest classifier. A Random Forest combines predictions from multiple decision trees to produce an overall result. During training, the trees learn patterns between player statistics and match outcomes. This allows the model to consider different combinations of information rather than basing the prediction on a single statistic.

An important part of the approach is feature engineering, where information from past matches is converted into numerical inputs. The features described in the project include player rankings and ranking points, recent form over the last five matches, performance on different court surfaces, tournament performance, previous results between the players, and performance against common opponents. These inputs give the model several ways to compare the players before predicting the result.

The project also uses a chronological split: earlier seasons are used for training and a later season is used for testing. According to the README, a prediction threshold of 0.48 was selected using 2018 validation data. This threshold is then used as part of the prediction process.

Tennis Match Prediction

4. Implementation Overview

The implementation is divided into separate parts for preparing data, creating features, training the model, making predictions, and providing a web interface. The `prepare_dataset.py` and `create_features.py` scripts are part of the data-preparation process. Additional feature scripts calculate specific player and match information, allowing the feature-generation work to be kept separate from the main training code.

The training scripts, including `train_model.py` and `train_extended_model.py`, are used to train model versions and evaluate their results. Once a model is trained, the saved model can be loaded by the prediction code rather than being trained every time a user requests a prediction. This makes it possible for the web application to use the trained model directly.

The web application uses Flask. The `app.py` file handles requests from the browser, while `src/predictor.py` contains the prediction logic. The `src/prediction_features.py` module prepares the selected match information in the format expected by the model. The application uses these features to produce a prediction and returns the result to the webpage.

The user interface is built with HTML, CSS, and JavaScript. It allows the user to enter or select match details and view the predicted outcome. The repository also includes `requirements.txt` for Python dependencies, setup and run scripts, and automated test files. These components support setting up the project, starting the application, and checking different parts of the implementation.

5. Results and Discussion

According to the project README, the model correctly predicted 1,779 out of 2,760 matches in the 2019 test set. This gives an accuracy of 64.46%. In other words, the model predicted the recorded winner in around 64 out of every 100 test matches. This is the result reported for the current implementation.

The result is below the reference figure of about 71%. The difference should be considered in the context of the available dataset and the modelling approach. The dataset used here ends in 2019, and differences in the number of matches, selected features, model settings, and training process can affect the final score. Accuracy also depends on the set of matches used for evaluation, so results should be compared only when the evaluation conditions are similar.

The reference system may also have used additional ATP Challenger match records and private datasets that were not available in our implementation, which could have contributed to its higher reported accuracy.

6. Conclusion

This project demonstrates how machine learning can be applied to tennis match prediction using historical ATP data. The implementation combines feature engineering, a Random Forest classifier, and a Flask-based web interface. Separate scripts handle data preparation and training, while the prediction modules connect the saved model to the application.

The current implementation reports 64.46% accuracy on the 2019 test set. The work provides a base that can be extended by using a larger and more up-to-date dataset, trying other classification models, and testing different feature combinations and training settings. Any changes can be compared on a separate test set to determine whether they improve prediction performance.